

LoanDesk

Three-person project & workload overview

CS3227 MP2 | Proposed team plan | Java SE 25 · JavaFX · Gradle

An equipment lending desktop app for a university lab, club or media room. Borrowers request equipment, supervisors authorize loans, and custodians manage physical equipment and fulfilment.

Three distinct responsibilities

Person / role	Owns	Key question
A - Borrower	Equipment discovery and borrowing requests	What do I need to borrow?
B - Lab Supervisor	Request review and authorization	Should this loan be approved?
C - Equipment Custodian	Inventory, collection and return	Can it be issued, and was it returned properly?

A typical loan

- A browses the catalogue and submits a request for one physical item and specific dates.
- B reviews the purpose, requests clarification if needed, and approves or rejects the request. Approval reserves the item.
- C records collection, then records the return and equipment condition.
- If returned damaged, C marks the item unavailable and completes basic maintenance before restoring it to service.

Working scope

Each request covers one physical item, such as Camera CAM-003. Start with preconfigured demonstration accounts, fixed equipment categories and basic maintenance notes. Payments, fines, notifications, QR scanning and multi-item requests are outside the initial scope.

Deployment assumption: users take turns signing into one shared desktop installation. Confirm this scope with the teaching team before committing; simultaneous use across separate computers needs additional infrastructure.

Ownership principle

Each person delivers their role end to end: JavaFX screens, service logic, automated tests and documentation. The allocations below are a starting agreement; rebalance after the first integrated loan works.

Person A | Borrower

Responsibility: help borrowers find equipment, submit requests and track loans.

Feature	Concrete implementation
Catalogue	Show names, categories, descriptions and condition.
Search and availability	Filter by name/category and dates using the shared availability service.
Submit request	Select one item; enter dates and purpose; validate input.
My Requests	Show own requests, statuses and decision reasons.
Clarification responses	Display the supervisor question; allow revision and resubmission.
Cancellation	Cancel own pending, clarification-required or approved requests before checkout.
My Loans	Show checked-out items, due dates and overdue indicators.
History	Show completed loans and return details.

Screens owned

Catalogue, Request Form, My Requests, Request Details and My Loans.

Shared engineering owned

- Login, logout and signed-in user/session handling.
- Navigation to the correct role interface.
- Common JavaFX components, including error dialogs and status labels.

Tests and documentation

Test: invalid request details; access to another borrower's records; edits after checkout; clarification and cancellation rules; catalogue filtering.

Document: borrower user guide, login/setup instructions, and UI/session design.

Agent skill and final delivery

Agent skill: UI acceptance testing. Check whether the agent identifies missing validation, unclear errors and incorrect displayed states from a feature specification.

Delivery coordination: product website and consistency of the assembled user guide.

Person B | Supervisor

Responsibility: review requests, authorize loans and enforce booking rules.

Feature	Concrete implementation
Review queue	Filter requests by status, date or borrower.
Request inspection	Show purpose, dates, equipment and relevant outstanding loans.
Clarification	Ask a question and send the request back to the borrower.
Resubmission review	Read the revised request and borrower response.
Approval	Check eligibility and availability; reserve the item for the dates.
Rejection	Require and record a reason.
Decision history	Record who decided, when and why.
Cancel approval	Cancel an approved, uncollected booking with a reason.

Screens owned

Review Queue, Review Details and Decision History.

Shared engineering owned

- Central role-permission mechanism and permission matrix.
- Availability and booking-conflict calculations used by A and B.
- Request-state rules: submission, clarification, approval and cancellation.
- CI configuration to build the app and run automated tests.

Tests and documentation

Test: overlapping approvals; availability rechecks; unavailable or retired items; unauthorized decisions; invalid transitions; reservation release on cancellation.

Document: supervisor user guide, permission matrix, request lifecycle and CI instructions.

Agent skill and final delivery

Agent skill: permission and workflow review. Evaluate it against controlled examples containing missing authorization or invalid transitions.

Delivery coordination: developer-guide assembly and submission checklist. A and C integrate the shared permission mechanism into their own service methods.

Person C | Custodian

Responsibility: maintain equipment records and fulfil approved loans.

Feature	Concrete implementation
Inventory	Add and edit physical equipment with unique IDs.
Service status	Mark items in service, unavailable, under maintenance or retired.
Collection queue	Show approved loans awaiting collection.
Checkout	Verify the item can be issued; record collection time.
Active loans	Show checked-out items and overdue indicators.
Returns	Record return time, condition and notes.
Basic maintenance	Record the issue, status and resolution; restore repaired items.
Fulfilment problems	Flag approved bookings affected by damage or outstanding loans.

Screens owned

Inventory, Equipment Details, Collections, Active Loans and Return/Maintenance Forms.

Shared engineering owned

- Storage/repository implementation, loading records and storage-error handling.
- Consistent saving and recovery from failed operations.
- Application error logging.

Tests and documentation

Test: checkout only after approval; no second checkout of an issued item; valid and non-duplicate returns; damaged-item restrictions; preserved retired-item history; restart persistence; save-failure consistency.

Document: custodian user guide, storage design, logging and recovery instructions.

Agent skill and final delivery

Agent skill: persistence-failure testing. Simulate failed saves and evaluate whether the tests detect inconsistent loan and equipment records.

Delivery coordination: Gradle release packaging and GitHub release. Keep maintenance simple: issue, status and resolution notes; exclude parts inventory and repair costs.

Shared implementation

Code ownership and integration

Component	Owner	Integration responsibility
Login, session, navigation	A	B and C connect their screens.
Borrower operations	A	Use agreed permission and transition rules.
Approval and request rules	B	A displays decisions; C fulfils approved bookings.
Availability calculations	B	A uses search results; C supplies equipment/loan state.
Permission mechanism	B	Everyone applies checks in their service methods.
Inventory and fulfilment	C	A and B use agreed interfaces to read equipment.
Persistence and logging	C	Everyone uses the shared infrastructure.
CI pipeline	B	Everyone keeps their tests passing.

Primary owner coordinates changes; others may contribute. Agree on data models and service interfaces before implementing separate screens.

Rules everyone must use

- Pending requests do not reserve equipment. Approval must recheck date conflicts and equipment service status.
- Only approved loans can be checked out; only checked-out loans can be returned. Overdue is calculated from the due date and outstanding return.
- Keep permission and workflow checks in services, not only in JavaFX buttons. Borrowers access only their own records.
- Retire equipment instead of deleting records referenced by past loans.
- Save related updates together. For a damaged return, the loan and equipment changes must either both succeed or leave a consistent prior state.

Decisions to settle at kickoff

Agree on date-boundary rules, collection timing, cancellation permissions, how staff handle affected bookings, the storage approach and the deployment assumption. Keep the agreed rules in the developer guide.

Team delivery plan

Integration milestones

Milestone	A delivers	B delivers	C delivers
1. Foundation	Login and role navigation	Permission matrix, lifecycle rules, CI	Storage foundation and sample equipment
2. First loan	Submit and view request	Approve or reject	Checkout and return
3. Role completion	Search, clarification, cancellation, history	Conflict checks, clarification, decision history	Inventory status, overdue view, maintenance
4. Reliability and release	Tests, guide, website	Workflow tests, developer guide, checklist	Storage tests, logging, release JAR

Everyone contributes

- Planning: agree scope, models, permissions and acceptance criteria.
- Testing: own feature tests plus complete cross-role integration tests.
- Review: A reviews B, B reviews C, C reviews A; involve affected teammates for shared-interface changes.
- Documentation: write your own user-guide and developer-guide sections.
- Agentic SE: define and evaluate one skill each; preserve instructions, outputs, mistakes and revisions.
- Logs: maintain verified summaries of your own prompts and agent interactions.
- Release: help test the packaged app across operating systems and check the guides against actual behaviour.

Required delivery items

Source code; Gradle-built release JAR with JavaFX dependencies; docs/UserGuide.md; docs/DeveloperGuide.md with acknowledgements; GitHub Pages product website; docs/Reflections.md with at least three detailed skill examples; verified interaction summaries under logs/.

Use the required public repository name CS3227-2610-MP2 within the team organization CS3227-2610-MP2-[project-name]. Keep master current for submission. Refer to the assignment for deadlines and final administrative requirements.

First shared acceptance check

A borrower submits a request, a supervisor approves it, and a custodian checks out and returns the item. Restart the app and verify that the complete history remains. Build this integrated journey before expanding every screen.